

Surrogates 7020

Chapter 4: Space-filling Design

Dr. Alex Bledar Konomi

Department of Mathematical Sciences
University of Cincinnati

Random Uniform

One simple option for design is random uniform. That is, fill a design matrix $\mathbf{X}_n$ in $[0, 1]^m$ with n runs via *runif*.

```
m <- 2
n <- 10
X <- matrix(runif(n*m), ncol=m)
colnames(X) <- paste0("x", 1:m)

plot(X, xlim=c(0,1), ylim=c(0,1))
```

* The trouble is, randomness is clumpy. With n of any reasonable size relative to input dimension m you're almost guaranteed to get design locations right next to one another, and thus gaps in other parts of the input space.

Can we do better?

- ▶ The probability of observing at least two points close to one other, as a relative share of the total input space (with $n = 10$ runs in $m = 2$ input dimensions), is quite high.
- ▶ *Latin hypercube samples* (LHSs) were created, or perhaps borrowed (more details below) from another literature, to alleviate this problem. The goal is to guarantee a certain degree of spread in the design.
- ▶ LHSs accomplish that by divvying the design region evenly into cubes, by partitioning coordinates marginally into equal-sized segments, and ensuring that the sample contains just one point in each such segment.
- ▶ In $2d$ the pattern of selected cubes (i.e., those containing a sample) resemble Latin squares (Design of Experiments) https://en.wikipedia.org/wiki/Latin_square.

Latin hypercube

A Latin hypercube (LH) of n runs for m factors is represented by an $n \times m$ matrix $L = (l_{ij})$. Each column l_j , for $j = 1, \dots, m$, contains a permutation of n equally spaced levels. For convenience, the n levels may be taken to be

$$-(n-1)/2, -(n-3)/2, \dots, (n-3)/2, (n-1)/2.$$

Right now levels span from $-(n-1)/2$ to $(n-1)/2$, so L spans $[-(n-1)/2, (n-1)/2]^m$. This is a mismatch to our design region $[0, 1]$ in m coordinates. But don't worry - we'll fix that with a normalization step momentarily.

Latin hypercube sample

A Latin hypercube sample (LHS), or sometimes design (LHD), X_n in the design space $[0, 1]^m$ is an $n \times m$ matrix with $(i, j)^{th}$ entry

$$x_{ij} = \frac{l_{ij} + (n - 1)/2 + u_{ij}}{n},$$

$i = 1, \dots, n, j = 1, \dots, m$, where the u_{ij} 's are independent uniform random deviates in $[0, 1]$. Observe that the denominator n normalizes so that x_{ij} is mapped into $[0, 1]^m$. The $(n - 1)/2$ term chooses a point in the corner of the selected square or (hyper) cube. Random adjustments u_{ij} place the j^{th} coordinate of run i elsewhere in the cube. If instead each u_{ij} is taken to be 0.5 the result is called a Latin sample, which will put x_{ij} right in the middle of one of the squares or (hyper) cubes.

LHS in Practice

The first step is to create the levels, which depend on the desired number of runs in the design, n .

```
l <- (-(n - 1)/2):((n - 1)/2)
```

Next, put m randomly permuted versions of the level vector l into a $n \times m$ matrix L .

```
L <- matrix(NA, nrow=n, ncol=m)
for(j in 1:m) L[,j] <- sample(l, n)
```

Finally, create a uniform jitter matrix U and combine with the level matrix L following a vectorized version of Eq. above.

```
U <- matrix(runif(n*m), ncol=m)
X <- (L + (n - 1)/2 + U)/n
```

LHS in a plot

```
plot(X, xlim=c(0,1), ylim=c(0,1), xlab="x1", ylab="x2")
abline(h=c((1 + (n - 1)/2)/n,1), col="grey", lty=2)
abline(v=c((1 + (n - 1)/2)/n,1), col="grey", lty=2)
```

LHS as a function

```
mylhs <- function(n, m)
{
  ## generate the Latin hypercube
  l <- (-(n - 1)/2):((n - 1)/2)
  L <- matrix(NA, nrow=n, ncol=m)
  for(j in 1:m) L[,j] <- sample(l, n)

  ## draw the random uniforms and turn the hypercube into a sample
  U <- matrix(runif(n*m), ncol=m)
  X <- (L + (n - 1)/2 + U)/n
  colnames(X) <- paste0("x", 1:m)

  ## return the design and the grid it lives on for visualization
  return(list(X=X, g=c((1 + (n - 1)/2)/n,1)))
}
```

LHS properties

- ▶ When using the new “library routine”, what can be expected in higher dimension m ?
- ▶ As we noticed in the example, LHSs are constructed so that there’s exactly one point in each of the n intervals $[0, 1/n), [1/n, 2/n), \dots, [(n-1)/n, 1)$, partitioning up each input coordinate, $j = 1, \dots, m$.
- ▶ This property is referred to as one-dimensional uniformity. As a consequence of that property, any projection into lower dimensions that can be obtained by dropping some of the coordinates will also be distributed uniformly.
- ▶ Therefore that lower m' -dimensional design will also be an LHS in the $m' < m$ dimensional unit hypercube.

LHS property in all pairs

```
Dlist <- mylhs(10, 3)

plot(Dlist$X[,1:2], xlim=c(0,1), ylim=c(0,1), xlab="x1", ylab="x2")
abline(h=Dlist$g, col="grey", lty=2)
abline(v=Dlist$g, col="grey", lty=2)

Is <- as.list(as.data.frame(combn(ncol(Dlist$X),2)))[-1]
par(mfrow=c(1,length(Is)))
for(i in Is) {
  plot(Dlist$X[,i], xlim=c(0,1), ylim=c(0,1),
       xlab=paste0("x", i[1]), ylab=paste0("x", i[2]))
  abline(h=Dlist$g, col="grey", lty=2)
  abline(v=Dlist$g, col="grey", lty=2)
}

$
```

Why LHS is good?

Popularity of LHSs as designs can largely be attributed to their role in reducing variance in numerical integration. Consider a function $y = f(x)$ where f is known, and x has a uniform distribution on $[0, 1]^m$, and where we wish to estimate $E\{Y\}$ via

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n f(x_i),$$

$$x_i \sim^{iid} U[0, 1]^m.$$

It turns out that if $f(x)$ is monotone in each coordinate of x then the variance of $\hat{\mu}$ is lower if an LHS is used rather than simple random sampling.

When LHS can be bad?

- ▶ There's no guarantee that the resulting design won't otherwise be somehow aliased, i.e., revealing patterns or clumpiness in other aspects of the variables' joint distribution.
- ▶ For example, the code chunk below repeatedly draws LHSs with $(n,m)=(10,2)$ (A counter is used to tally the number of attempts before that criterion is met).

```
count <- 0
while(1) {
  count <- count + 1
  Dlist <- mylhs(10, 2)
  o <- order(Dlist$X[,1])
  x <- Dlist$X[o,2]
  if(all(x[1:9] < x[2:10] + 1/20)) break
}
```

Using inverse CDF

LHSs may be extended to marginals **other than uniform** through a simple inverse cumulative distribution function (CDF) transformation. The steps are:

- ▶ Generate an ordinary LHS, then
- ▶ apply an inverse CDF (quantile function) of your choice on each of the marginals.

This will cause the marginals to take on distributions with those CDFs, but at the same time ensure that there's not “more clumpiness than necessary” in the joint distribution.

Beta distributed marginals

Utilize beta distributed marginals, of which uniform is a special case. Parameters α and β are specified as m -vectors shape1 and shape2 following the nomenclature used by *qbeta*,

```
mylhs.beta <- function(n, m, shape1, shape2){
  ## generate the Latin Hypercube and turn it into a sample
  l <- (-(n - 1)/2):((n - 1)/2)
  L <- matrix(NA, nrow=n, ncol=m)
  for(j in 1:m) L[,j] <- sample(l, n)
  U <- matrix(runif(n*m), ncol=m)
  X <- (L + (n - 1)/2 + U)/n
  ## calculate the grid for that design
  g <- (L + (n - 1)/2)/n
  g <- rbind(g, 1)
  for(j in 1:m) { ## redistribute according to beta quantiles
    X[,j] <- qbeta(X[,j], shape1[j], shape2[j])
    g[,j] <- qbeta(g[,j], shape1[j], shape2[j])
  }
  colnames(X) <- paste0("x", 1:m)
  ## return the design and the grid it lives on for visualization
  return(list(X=X, g=g))
}
```

Beta distributed marginals

```
Dlist <- mylhs.beta(10, 2, shape1=c(3,1/2), shape2=c(2,1/2))

plot(Dlist$X, xlim=c(0,1), ylim=c(0,1), xlab="x1", ylab="x2")
abline(v=Dlist$g[,1], col="grey", lty=2)
abline(h=Dlist$g[,2], col="grey", lty=2)
```

To look closer at the marginals, code below generates a much larger LHS.

```
X <- mylhs.beta(1000, 2, shape1=c(3,1/2), shape2=c(2,1/2))$X
par(mfrow=c(1,2))
x <- seq(0,1,length=100)
hist(X[,1], main="", xlab="x1", freq=FALSE)
lines(x, dbeta(x, 3, 2), col=2)
hist(X[,2], main="", xlab="x2", freq=FALSE)
lines(x, dbeta(x, 1/2, 1/2), col=2)
```

LHSs fewer samples

- ▶ Designs generated in this way are useful for input sensitivity analyses and related exercises common in the uncertainty quantification (UQ) literature.
- ▶ An important aspect in both settings involves exploring how a distribution on inputs propagates to a distribution of outputs through an opaque blackbox apparatus.
- ▶ Random sampling can facilitate a study of main effects and second-order uncertainty indices for nominal inputs under random jitter, or for inputs concentrated in a typical regime.
- ▶ LHSs offer similar functionality while requiring many fewer samples be pushed through a computationally intensive blackbox, say one constructed as a daisy-chain of numerical solvers, surrogates, and so on.

LHS for assessment of generalization error

- ▶ LHSs are also handy in synthetic predictive benchmarking exercises emphasizing assessment of generalization error. Sometimes these are called “bakeoffs” or “horse races”.
- ▶ The idea is to generate training and out-of-sample testing sets where input–output pairs from the former are used to build fitted models of several varieties, and ones from the latter are held out to subsequently compare predictors derived from those fitted models.
- ▶ Inputs from an LHS offer certain guarantees on spread, and when collected simultaneously for training and testing can yield a partition where out-of-sample measurements are made on novel (but not too novel) input settings.

Illustration

To illustrate, consider the following partition of an $n = 20$. LHS in 2d where training inputs come from the first ten, and testing from the last ten.

```
Dlist <- mylhs(20, 2)
Xtrain <- Dlist$X[1:10,]
Xtest <- Dlist$X[11:20,]

plot(Xtrain, xlim=c(0,1), ylim=c(0,1), xlab="x1", ylab="x2")
points(Xtest, pch=20)
abline(v=Dlist$g, col="grey", lty=2)
abline(h=Dlist$g, col="grey", lty=2)
```

LHS Bakeoff

1. Generate an LHS of size $n + n'$ creating X .
2. Evaluate the true response at all $n + n'$ sites, yielding $Y \sim f(X)$, and combine them with inputs to obtain $n + n'$ data pairs $D = (X, Y)$.
3. Randomly partition into n training pairs $T_n = (X_n^t, Y_n^t)$ and n' testing or validation pairs $V_{n'} = (X_{n'}^v, Y_{n'}^v)$.
 - Since rows of the LHS are random, as from *mylhs*, it's usually sufficient to take the first n and last n' pairs, respectively.
4. Train K competing methods on T_n yielding predictors $\hat{y}^{(k)}(\cdot)|T_n$, for $k = 1, \dots, K$. Test out-of-sample by making predictions on the validation set $V_{n'}$, saving them as $\hat{Y}_{n'}^{(k)} = \hat{y}^{(k)}(X_{n'}^v)$.

LHS Bakeoff - cont

5. Test out-of-sample by making predictions on the validation set $V_{n'}$, saving them as $\hat{Y}_{n'}^{(k)} = \hat{y}^{(k)}(X_{n'}^v)$.
6. Calculate metrics $m_r^{(k)}$ for each method $k = 1, \dots, K$ comparing $\hat{Y}_{n'}^{(k)}$ to the held-out values $Y_{n'}^v$. Examples include
 - ▶ proper scoring rule (See sec. 5.2 and Gneiting and Raftery 2007),
 - ▶ root mean-squared (prediction) error (RMSE) (or mean absolute error (MAE))

$$m_r^{(k)} = \sqrt{\frac{1}{n} \sum_{i=1}^{n'} (y_i^u - \hat{y}_i^{(k)})^2}$$

Return one or more $R \times K$ matrices M comprised of $m_r^{(k)}$ -values calculated in Step 6, above, and/or more compact summaries of their empirical distribution in order to determine the winner of the bakeoff.

Maximin designs

- ▶ If what we want is points spread out, then perhaps it makes sense to design criteria that deliberately spreads points out!
- ▶ For that we need a notion of distance by which to measure spread. The development here takes the simple choice of (squared) Euclidean distance

$$d(x, x') = \|x - x'\|^2 = \sum_{j=1}^m (x_j - x'_j)^2$$

- ▶ the framework is otherwise quite general if other choices, such as say Mahalanobis distance, were desired instead.
- ▶ A design $X_n = x_1, \dots, x_n$ which maximizes the minimum distance between all pairs of points,

$$X_n = \operatorname{argmax}_{X_n} \min_{i \neq k} d(x_i, x_k) : i, k = 1, \dots, n$$

is called a *maximin design*.

Maximin in Practice

- ▶ Maximin designs: easy to say, not so easy to do. The details are all in the algorithm.
- ▶ The leading $\max X$ is effectively a maximization over $n \times m$ “dimensions”. Potential design sites $x_1, \dots, x_n$ making up $\mathbf{X}_n$ each occupy m coordinates in the input space. Every $\mathbf{X}_n$ entertained must be evaluated under a criterion that entails at best $O(n^2)$ operations, at least at first glance. Although the criterion suggests a deterministic solution for a particular choice of n and m , in fact most solvers are stochastic, owing to the challenge of the optimization setting.
- ▶ A naive but simple iterative algorithm entails proposing a random change to one of the n sites x_i , and accepting or rejecting that change by consulting the criterion. Since only $n - 1$ distances change under such a swap, a clever implementation can get away with $\mathcal{O}(n)$ rather than $\mathcal{O}(n^2)$ calculations.

Maximin in Practice

- ▶ Having a fast (Euclidean) distance calculation, not just for one pair of points but potentially for many, is crucial to creating maximin designs efficiently.
- ▶ R has a *dist* function which is appropriate.
- ▶ For slightly simpler implementation and to connect better to GP code in later chapters (particularly Chapter 5), we shall instead borrow distance from the plgp package (Gramacy 2014) on CRAN.
- ▶ The distance function allows distances to be calculated between pairs of coordinates, from one coordinate to many, and from many to many, where the for loops are optimized in compiled C code.

Maximin in Practice

```
library(plgp)

n=10; m=2;
distance(X[1:2,], X[3:5,])

X1 <- matrix(runif(n*m), ncol=m)
dX1 <- distance(X1)
dX1 <- dX1[upper.tri(dX1)]
md1 <- min(dX1)
X2 <- matrix(runif(n*m), ncol=m)
dX2 <- distance(X2)
dX2 <- dX2[upper.tri(dX2)]
md2 <- min(dX2)

plot(X1, xlim=c(0,1.25), ylim=c(0,1.25), xlab="x1", ylab="x2")
points(X2, pch=20)
legend("topright", paste("md =", round(c(md1, md2), 5)), pch=c(21,20))
```

Stochastic maximize minimum distance

- ▶ Iterative pairwise comparisons of designs of size n , as in the example above, would be quite cumbersome computationally.
- ▶ Yet the simplicity of this idea makes it attractive from an implementation perspective. The coding effort is just as easy, but more efficient computationally, if modified to consider pairs X_1 and X_2 which differ by just one point.
- ▶ Along those lines, code below sets up a simple stochastic exchange algorithm to maximize minimum distance, starting from a random uniform design (X_1 above), proposing a random row to swap out, and a new set of random coordinates to swap back in (thus implicitly creating X_2).
- ▶ If the resulting maximin distance is not improved, then the proposed exchange is undone: a rejection. Otherwise the implicit X_2 is accepted as the new X_1 . After accepting or rejecting 10,000 such proposals, the algorithm terminates.

Maximin in Practice

```
T <- 10000
for(t in 1:T) {
  row <- sample(1:n, 1)
  xold <- X1[row,]      ## random row selection
  X1[row,] <- runif(m)   ## random new row
  d <- distance(X1)
  d <- d[upper.tri(d)]
  mdprime <- min(d)
  if(mdprime > md1) {    ## accept
    md1 <- mdprime
  } else {               ## reject
    X1[row,] <- xold
  }
}
```

What do we notice?

- ▶ Design sites are nice and spread out, but they're pushed to boundaries which may be undesirable. That's an obvious consequence of the search criteria. Points are pushed to the edge of the input space because they're repelled away from other ones nearby.
- ▶ As input dimension m increases this boundary effect becomes more extreme. Surface area will eventually dwarf interior volume in relative terms. Also notice a lack of one-dimensional uniformity here. Marginally, projecting over either of x_1 or x_2 , there appears to be just three or four truly unique settings.

Maximin in Code

```
mymaximin <- function(n, m, T=100000)
{
  X <- matrix(runif(n*m), ncol=m)      ## initial design
  d <- distance(X)
  d <- d[upper.tri(d)]
  md <- min(d)

  for(t in 1:T) {
    row <- sample(1:n, 1)
    xold <- X[row,]                    ## random row selection
    X[row,] <- runif(m)                ## random new row
    d <- distance(X)
    d <- d[upper.tri(d)]
    mdprime <- min(d)
    if(mdprime > md) { md <- mdprime   ## accept
    } else { X[row,] <- xold }         ## reject
  }

  return(X)
}
```

Generating a maximin design

Now how about generating a maximin design with $n=10$ and $m=3$?

```
X <- mymaximin(10, 3)
```

```
Is <- as.list(as.data.frame(combn(ncol(X),2)))
par(mfrow=c(1,length(Is)))
for(i in Is) {
  plot(X[,i], xlim=c(0,1), ylim=c(0,1), type="n", xlab=paste0("x", i[1]),
       ylab=paste0("x", i[2]))
  text(X[,i], labels=1:nrow(X))
}
```

Some negative properties of Maximin

- ▶ Things don't look so good here, at least compared to similar plots for the $3d$ LHS. Of course, it's harder to visualize a $3d$ design via $2d$ projections.
- ▶ **A push to boundaries, near non-uniqueness of values in the input coordinates, etc.**
- ▶ Some $2d$ projections may have close pairs, but other panels indicate that those points are far apart in their other coordinates.
- ▶ In fact, those which are among the closest in one input pair tend to be among the farthest in the others. **It's abundantly clear that a projection of a maximin design is not itself maximin in the lower dimension.** There's no one-dimensional uniformity.

Some negative properties of Maximin

- ▶ Maximin selection naturally extends to sequential design. That is, we may condition on an existing design $X_n^{orig} \equiv x_{orig}$ and ask what new runs could augment that design to produce $X_{n'} \equiv x$ where the new runs optimize the maximin criterion.
- ▶ Only elements of x are being chosen, but the desired settings of its coordinates are spaced out relative to themselves, and to existing locations in x_{orig} . Mathematically,

$$X_{n'} = \operatorname{argmax}_{X_{n'}} \min d(x_i, x_k) : i = 1, \dots, n'; k \neq i = 1, \dots, n' + n,$$

where X_n^{orig} is comprised of n locations indexed as $x_{n'+1}, \dots, x_{n'+n}$. Sometimes $X_{n'}$ in this context is called an augmenting design.

Adapting our *mymaximin* function

```
mymaximin <- function(n, m, T=100000, Xorig=NULL)
{ X <- matrix(runif(n*m), ncol=m)      ## initial design
  d <- distance(X); d <- d[upper.tri(d)];
  md <- min(d)
  if(!is.null(Xorig)) {                ## new code
    md2 <- min(distance(X, Xorig))
    if(md2 < md) md <- md2
  }
  for(t in 1:T) {
    row <- sample(1:n, 1)
    xold <- X[row,]                    ## random row selection
    X[row,] <- runif(m)                ## random new row
    d <- distance(X); d <- d[upper.tri(d)]; mdprime <- min(d);
    if(!is.null(Xorig)) {              ## new code
      mdprime2 <- min(distance(X, Xorig))
      if(mdprime2 < mdprime) mdprime <- mdprime2
    }
    if(mdprime > md) { md <- mdprime    ## accept
    } else { X[row,] <- xold }          ## reject
  }
  return(X)
}
```

Sequential mymaximin in Practice

Let's see how it works. Below we ask for $n' = 5$ new runs, augmenting our earlier $n = 10$ sites stored in

```
X2 <- mymaximin(5, 2, Xorig=X1)
```

```
plot(X1, xlim=c(0,1), ylim=c(0,1), xlab="x1", ylab="x2")
points(X2, pch=20)
```

```
X2 <- mymaximin(5, 3, Xorig=X)
X <- rbind(X2, X)
```

```
Is <- as.list(as.data.frame(combn(ncol(X),2)))
par(mfrow=c(1,length(Is)))
for(i in Is) {
  plot(X[,i], xlim=c(0,1), ylim=c(0,1), type="n", xlab=paste0("x", i[1]),
    ylab=paste0("x", i[2]))
  text(X[,i], labels=1:nrow(X), col=c(rep(2,5), rep(1,10)))
}
```

3D representation

Sometimes the illusion of perspective offered by the *scatterplot3d* library is helpful in such inspections.

```
library(scatterplot3d)
scatterplot3d(X, type="h", color=c(rep(2,5), rep(1,10)),
  pch=c(rep(20,5), rep(21,10)), xlab="x1", ylab="x2", zlab="x3")
```

Libraries and hybrids

- ▶ Our *mylhs* function from above, and its extension *mylhs.beta* are hard to improve upon in terms of computational efficiency. Nevertheless it's simpler to rely on canned alternatives.
- ▶ Similar functionality is offered by *randomLHS* in the *lhs* library (Carnell 2018) on CRAN. Although tailored to uniform marginals, that library does offer a sequential alternative in *augmentLHS*.
- ▶ The *lhs* package also provides a nice hybrid between *maximin* and *LHS* in *maximinLHS* which helps avoid aliasing problems of both ordinary LHS and maximin designs. Think of this as entertaining candidate LHSs and preferring ones whose minimum distance between design element pairs is large (MD Morris and Mitchell 1995).

Libraries and hybrids

Typical search methods proceed similarly to our sequential maximin method, described above, usually involving stochastic exchange of pairs of levels, separately (and possibly also randomly) in each coordinate, in order to preserve a Latin square structure. To compare/contrast with some of our earlier examples, code below builds one of these hybrid designs.

maximinLHS Example

```
library(lhs)
X <- maximinLHS(10, 2)

plot(X, xlim=c(0,1), ylim=c(0,1), xlab="x1", ylab="x2")
abline(h=seq(0,1,length=11), col="gray", lty=2)
abline(v=seq(0,1,length=11), col="gray", lty=2)
```

maximinLHS Properties

- ▶ Since it's still an LHS, points are not pushed to the boundaries.
- ▶ One-dimensional uniformity guarantees at most one point near each edge.
- ▶ Consequently such designs are often more desirable than an LHS or maximin on their own, although there are reasons to prefer these original, raw (un-hybridized), options.
- ▶ I'm not aware of any software offering a sequential update to a hybrid maximin LHS design.

- ▶ Related, so-called maximum projection design (Joseph, Gul, and Ba 2015), extends the LHS one-dimensional uniformity property to larger subspaces.
- ▶ Building a maxpro design entails calculations similar to those involved in maximin, and like maximin they emit a natural sequential analog. A package is available on CRAN called MaxPro (Ba and Joseph 2018).
- ▶ Sometimes computer experiments contain factor-valued inputs – i.e., categorical rather than real-valued ones. Maxpro designs have recently been extended to accommodate this case. Maximin “sliced” LHD (SLHD) hybrids (Ba, Myers, and Brenneman 2015) also represent an attractive option in this setting. See SLHD (Ba 2015) on CRAN.